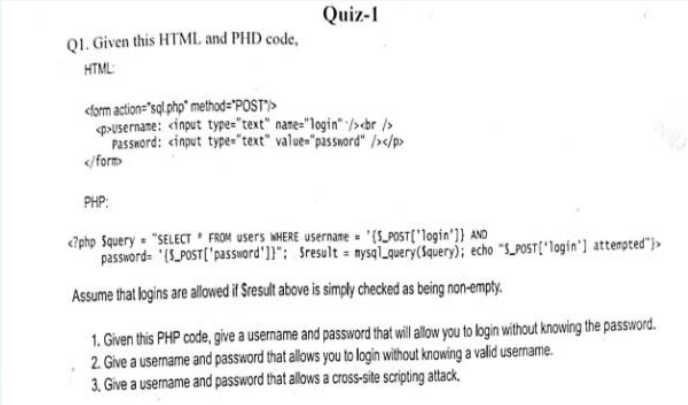
SQL Injection Class Test



Quiz-1

Q1. Given this HTML and PHD code,

HTML:

```
<form action="sql.php" method="POST"/>
  <p>username: <input type="text" name="login" /><br />
     Password: <input type="text" value="password" /></p>
</form>
```

PHP:

```
<?php $query = "SELECT * FROM users WHERE username = '{$_POST['login']} AND
      password= '{$_POST['password']}'";  $result = mysql_query($query); echo "$_POST['login'] attempted"?>
```

Assume that logins are allowed if $result above is simply checked as being non-empty.

1. Given this PHP code, give a username and password that will allow you to login without knowing the password.
2. Give a username and password that allows you to login without knowing a valid username.
3. Give a username and password that allows a cross-site scripting attack.

how to solve Q1. question 1 (explain properly)

This question is testing your understanding of **SQL Injection** (and a bit of XSS). I'll explain it clearly so you can write it in exam format.

🔍 First, understand what the code is doing

HTML issue

- `username` field → has name `"login"` ✅
- `password` field → **NO name attribute** ❌

  ```
  <input type="text" value="password" />
  ```

👉 So `$_POST['password']` will be **empty / undefined**

PHP query (very important)

```
$query = "SELECT * FROM users WHERE username = '{$_POST['login']}'
AND password = '{$_POST['password']}'";
```

👉 User input is directly inserted into SQL → **no validation** → **vulnerable to SQL Injection**

👉 Login condition:

> Login succeeds if `$result` is **non-empty**

✅ Q1: Login WITHOUT knowing password

💡 Idea:

Since password is broken (not sent properly), we bypass the condition using SQL injection.

✔️ Concept:

Make the WHERE condition always TRUE.

✔️ Example input:

- Username:

  ```
  ' OR '1'='1
  ```

- Password:
  (anything / empty)

✔️ Resulting query:

```
SELECT * FROM users
WHERE username = '' OR '1'='1' AND password = ''
```

👉 `'1'='1'` is always TRUE
👉 So query returns rows → login succeeds

✍️ Exam Answer:

- Use SQL injection to make condition always true
- Example: username = `' OR '1'='1`
- This bypasses password checking

✅ Q2: Login WITHOUT knowing username

💡 Idea:

Same trick — bypass both username and password conditions

✔️ Example input:

- Username:

  ```
  ' OR 1=1 --
  ```

- Password:
  anything

✔️ Result:

```
SELECT * FROM users
WHERE username = '' OR 1=1 -- ' AND password = ''
```

👉 `--` comments out rest of query
👉 `OR 1=1` → always TRUE
👉 Login successful without valid username

✍️ Exam Answer:

- Use SQL injection with comment (`--`)
- Example: `' OR 1=1 --`
- This ignores both username and password checks

✅ Q3: Cross-Site Scripting (XSS)

💡 Look at this line:

```
echo "$_POST['login'] attempted";
```

👉 User input is printed **without sanitization**

✔️ Idea:

Inject JavaScript into username

✔️ Example:

- Username:

  ```
  <script>alert('XSS')</script>
  ```

✔️ Output:

```
<script>alert('XSS')</script> attempted
```

👉 Browser executes script → XSS attack

✍️ Exam Answer:

- Input malicious script in username
- Example: `<script>alert('XSS')</script>`
- Script executes because input is not sanitized

🧠 Final Summary (write this nicely)

Part	Concept	Attack Type
Q1	Bypass password	SQL Injection
Q2	Bypass username + password	SQL Injection
Q3	Inject script in output	XSS

⚠️ Key Learning Points

- Never directly use user input in SQL queries
- Always use:
  - Prepared statements
  - Input validation
  - Output encoding

If you want, I can convert this into a **perfect 6-mark exam answer format** (like your lab style).

Q2. Consider the following login page:

```
set ok = execute( "SELECT * FROM Users
     WHERE user=' "  &  form("user")  & " '
     AND   pwd=' " & form("pwd") & " '" ):

if not ok.EOF
     login success
else  fail;
```

Is this exploitable? If so, give possible SQL injection attacks with this code.

Yes — **this code is definitely exploitable**. It is vulnerable to **SQL Injection** because user input is directly concatenated into the query without validation.

🔍 Step 1: Understand the query

The code effectively builds this SQL:

```
SELECT * FROM Users
WHERE user = '<input_user>'
AND pwd = '<input_pwd>'
```

👉 `form("user")` and `form("pwd")` come directly from user input
👉 No sanitization / escaping ❌
👉 This is classic SQL injection vulnerability

⚠️ Why it is exploitable

- User controls both `user` and `pwd`
- Query structure can be **modified**
- Login success condition:

  `if not ok.EOF → success`

👉 Means: if query returns **any row**, login succeeds

✅ Possible SQL Injection Attacks

🔹 1. Bypass Authentication (Always TRUE)

✔️ Input:

- Username:

  ```
  ' OR '1'='1
  ```

- Password:

  ```
  ' OR '1'='1
  ```

✔️ Query becomes:

```
SELECT * FROM Users
WHERE user = '' OR '1'='1'
AND pwd = '' OR '1'='1'
```

👉 `'1'='1'` is always TRUE
👉 Query returns rows → login success

🔹 2. Using Comment to Ignore Password

✔️ Input:

- Username:

  ```
  ' OR 1=1 --
  ```

- Password:
  anything

✔️ Query:

```
SELECT * FROM Users
WHERE user = '' OR 1=1 -- '
AND pwd = ''
```

👉 `--` comments out password condition
👉 `OR 1=1` makes condition TRUE
👉 Login successful

🔹 3. Login as Specific User (e.g., admin)

✔️ Input:

- Username:

  ```
  admin' --
  ```

- Password:
  anything

✔️ Query:

```
SELECT * FROM Users
WHERE user = 'admin' -- '
AND pwd = ''
```

👉 Password check removed
👉 Logs in as **admin**

🔹 4. Blind SQL Injection (Conceptual)

Even if results are hidden, attacker can test conditions:

✔️ Input:

```
' OR SUBSTRING(user,1,1)='a' --
```

👉 Helps attacker **guess data step by step**

✍️ Final Exam Answer (write like this)

Yes, the code is exploitable due to SQL Injection.
User inputs are directly concatenated into the SQL query without sanitization.

Possible attacks:

1. **Authentication bypass**
   - Input: `' OR '1'='1`
   - Makes condition always true → login success
2. **Comment-based injection**
   - Input: `' OR 1=1 --`
   - Comments out password check → bypass authentication
3. **Login as specific user**
   - Input: `admin' --`
   - Removes password condition → logs in as admin

🧠 Key Point (important for marks)

👉 Root cause:

> **Lack of input validation and use of dynamic SQL queries**

👉 Prevention:

- Prepared statements (parameterized queries)
- Input sanitization
- ORM frameworks

If you want, I can convert this into a **perfect 6-mark structured answer (intro + diagram + conclusion)**.

Q3. Consider the following code for a stored procedure (which is accessible to 'public' by default) which executes a search query for a web application. In case no search query is provided, the query results the details of all the products in the database. If a query is provided, it returns all products which have the search term in the product name.

```
CREATE PROCEDURE SP_EmployeeSearch @empname varchar(100) = NULL AS
DECLARE @sql nvarchar(4000)
SELECT @sql = ' SELECT EmpID, EmpName, Position, Salary ' +
' FROM Employee Where '
IF @EmpID IS NOT NULL
SELECT @sql = @sql + ' EmpName LIKE '' + @empname + ''
EXEC (@sql)
```

Is the stored procedure vulnerable to SQL injection? Why or why not?

✅ Q3 Answer (SQL Injection in Stored Procedure)

🔍 Step 1: Understand the Procedure

The stored procedure dynamically builds a query:

```
SELECT @sql =
'SELECT EmpID, EmpName, Position, Salary
 FROM Employee WHERE '

IF @EmpID IS NOT NULL
SELECT @sql = @sql +
' EmpName LIKE ''' + @empname + ''' '

EXEC(@sql)
```

👉 Key point:

- The query is constructed using **string concatenation**
- Then executed using **`EXEC(@sql)`**

⚠️ Is it vulnerable?

✅ YES — It is vulnerable to SQL Injection

❗ Why?

1. Direct concatenation of user input

- `@empname` is directly added into query
- No validation / escaping ❌

---

2. Dynamic SQL execution

- `EXEC(@sql)` executes the constructed string
- Any malicious input becomes part of SQL

---

3. No parameterization

- Proper stored procedures should use parameters
- This one behaves like raw SQL ❌

💣 Example SQL Injection Attacks

🔹 1. Return all records

Input:

```
%' OR 1=1 --
```

Query becomes:

```
SELECT EmpID, EmpName, Position, Salary
FROM Employee
WHERE EmpName LIKE '%%' OR 1=1 --'
```

👉 `OR 1=1` → always TRUE
👉 Returns all records

🔹 2. Data Extraction / Manipulation

Input:

```
%' UNION SELECT username, password, null, null FROM Users --
```

Query:

```
SELECT EmpID, EmpName, Position, Salary
FROM Employee
WHERE EmpName LIKE '%%'
UNION SELECT username, password, null, null FROM Users --'
```

👉 Attacker can extract sensitive data

🔹 3. Destructive Query (if permissions allow)

Input:

```
%'; DROP TABLE Employee; --
```

Query:

```
SELECT EmpID, EmpName, Position, Salary
FROM Employee
WHERE EmpName LIKE '%'; DROP TABLE Employee; --'
```

👉 Can delete table ❗

✍️ Final Exam Answer (write this)

Yes, the stored procedure is vulnerable to SQL injection.

Reasons:

1. Uses **dynamic SQL (`EXEC(@sql)`)**
2. User input (`@empname`) is **directly concatenated**
3. No input validation or parameterization

Example attack:

- Input: `%' OR 1=1 --`
- This makes condition always true and returns all records

🛡️ Prevention (important for marks)

- Use **parameterized queries**
- Use `sp_executesql` with parameters
- Validate and sanitize input

🧠 One-line conclusion

> Dynamic SQL + direct user input = SQL Injection vulnerability

---

If you want, I can give you a **perfect 6-mark answer format with headings + keywords (for max marks)**.

Q4. Given any XACML policy and one example access request for your XML DB. Considering this policy and access request, determine the access response.

This question is **open-ended** → you must **create your own XACML policy + request + decision**.

I'll give you a **perfect exam-ready answer** you can directly write.

✅ Q4 Answer (XACML Policy + Request + Decision)

🔐 1. XACML Policy

Assume a policy:

👉 *"Only users with role = admin can read employee data"*

```
<Policy>
  <Target>
    <Resource>EmployeeData</Resource>
    <Action>read</Action>
  </Target>

  <Rule Effect="Permit">
    <Condition>
      subject.role == "admin"
    </Condition>
  </Rule>

  <Rule Effect="Deny"/>
</Policy>
```

📩 2. Access Request

Case 1: Non-admin user

```
<Request>
  <Subject>
    <Attribute Name="role">user</Attribute>
  </Subject>
  <Resource>EmployeeData</Resource>
  <Action>read
</Request>
```

⚖️ 3. Access Decision

- Role = **user**
- Policy requires role = **admin**

👉 Condition fails ❌

✅ Result:

```
Decision: DENY
```

📩 4. Another Request (for clarity)

Case 2: Admin user

```
<Request>
  <Subject>
    <Attribute Name="role">admin</Attribute>
  </Subject>
  <Resource>EmployeeData</Resource>
  <Action>read</Action>
</Request>
```

⚖️ 5. Decision

- Role = **admin**
- Condition satisfied ✅

✅ Result:

```
Decision: PERMIT
```

✍️ Final Conclusion (write this)

- XACML evaluates access requests based on defined policies
- If conditions match → **Permit**
- Otherwise → **Deny**

🧠 Simple one-line memory trick

👉 **Match → Permit | No match → Deny**

If you want, I can give you a **short 5-line version for writing quickly in exam**.